

Python Interview Questions

Decorators & Closures — Top 100

Closures · LEGB · functools · Class decorators · Async · Real-world Patterns

Section 1 — Closures & Free Variables (Q1–Q20)

Q1. What is a closure in Python?

A closure is a function that remembers the variables from its enclosing scope even after that scope has finished executing. It 'closes over' the free variables it references, keeping them alive in a cell object.

```
def make_adder(n):
    def adder(x):          # adder closes over 'n'
        return x + n
    return adder

add5 = make_adder(5)
add10 = make_adder(10)

print(add5(3))           # 8
print(add10(3))          # 13
print(add5.__closure__[0].cell_contents) # 5
```

Q2. What are free variables?

Free variables are names used inside a function that are neither local to that function nor global — they come from an enclosing scope. Python stores them in cell objects accessible via `__closure__`.

```
def outer():
    x = 10                # x is free in 'inner'
    def inner():
        return x          # uses enclosing x
    return inner

fn = outer()
print(fn.__code__.co_freevars) # ('x',)
print(fn.__closure__[0].cell_contents) # 10
```

Q3. What is the LEGB rule?

Python resolves names in four scopes in order: Local (inside the current function), Enclosing (any enclosing function scopes, inside-out), Global (module level), Built-in (builtins module). The first match wins.

```
x = 'global'

def outer():
    x = 'enclosing'
    def inner():
        # x = 'local'    # if present, local wins
        print(x)        # finds 'enclosing'
    inner()

outer()    # enclosing
print(x)   # global
```

Q4. What is the late-binding closure problem?

Closures bind to variable names, not values. In a loop, all closures share the same variable which holds its final value once the loop ends. This is the most common closure gotcha.

```
# BUG: all functions return 4 (i's final value)
fns = [lambda: i for i in range(5)]
print([f() for f in fns])    # [4, 4, 4, 4, 4]

# FIX 1: capture with default argument
fns = [lambda i=i: i for i in range(5)]
print([f() for f in fns])    # [0, 1, 2, 3, 4]

# FIX 2: factory function
def make(i): return lambda: i
fns = [make(i) for i in range(5)]
print([f() for f in fns])    # [0, 1, 2, 3, 4]
```

Q5. What is the nonlocal keyword and why is it needed?

Without nonlocal, assigning to a name inside a nested function creates a new local variable, shadowing the enclosing one. nonlocal declares that the name refers to the nearest enclosing scope's variable, allowing rebinding.

```
def make_counter():
    count = 0
    def increment():
        nonlocal count    # without this: UnboundLocalError
        count += 1
        return count
    return increment

counter = make_counter()
print(counter())    # 1
print(counter())    # 2
print(counter())    # 3
```

Q6. What is the difference between nonlocal and global?

global refers to the module-level namespace. nonlocal refers to the nearest enclosing function scope (not global). Use global to rebind a module variable; use nonlocal to rebind a variable in an outer function.

```
x = 'module'

def outer():
    x = 'outer'
    def inner():
        nonlocal x
        x = 'inner_modified'
    inner()
    print(x)      # 'inner_modified'

outer()
print(x)          # 'module' - global unchanged
```

Q7. How do you inspect a closure's captured variables?

Use `fn.__closure__` to access the tuple of cell objects, and `cell.cell_contents` to read each value. `fn.__code__.co_freevars` lists their names in the same order.

```
def make_multiplier(factor, label):
    def multiply(x):
        return f'{label}: {x * factor}'
    return multiply

fn = make_multiplier(3, 'triple')

names = fn.__code__.co_freevars
values = [c.cell_contents for c in fn.__closure__]
print(dict(zip(names, values)))
# {'factor': 3, 'label': 'triple'}
```

Q8. Can a closure modify a mutable object from the enclosing scope without nonlocal?

Yes. nonlocal is needed only for rebinding (assignment). Mutating a mutable object (list, dict) does not require nonlocal because you are changing the object's contents, not the variable binding.

```
def make_appender():
    items = []
    def append(x):
        items.append(x)    # mutation - no nonlocal needed
        return items
    return append

app = make_appender()
print(app(1))    # [1]
print(app(2))    # [1, 2]
```

Q9. What is a factory function and how does it use closures?

A factory function returns a new function customised by the arguments passed to the factory. The returned function closes over those arguments, creating independent, parameterised callables.

```
def power_factory(exp):
    def power(base):
        return base ** exp
    return power

square = power_factory(2)
cube   = power_factory(3)

print(square(4))    # 16
print(cube(3))      # 27
print(square is cube) # False – independent callables
```

Q10. How does Python implement closures at the bytecode level?

Python uses `LOAD_DEREF` / `STORE_DEREF` opcodes for cell variables and free variables. The compiler detects that a variable is shared between an inner and outer function and allocates a cell object on the heap instead of the stack, so it survives after the outer frame is gone.

```
import dis

def outer():
    x = 42
    def inner(): return x
    return inner

dis.dis(outer())    # shows LOAD_DEREF for x
# LOAD_DEREF  0 (x)  <- reads from cell object
```

Q11. What is `__closure__` and what does `None` mean?

`__closure__` is a tuple of cell objects for a function's free variables. It is `None` when the function has no free variables (i.e., it is not a closure).

```
def plain(): return 42
print(plain.__closure__)    # None – not a closure

def outer():
    x = 1
    def inner(): return x
    return inner

fn = outer()
print(fn.__closure__)        # (<cell at 0x...>,)
print(type(fn.__closure__[0])) # <class 'cell'>
```

Q12. Can closures be used for memoization?

Yes. A closure over a dict provides a simple, thread-unsafe memoization cache without needing a class or global variable.

```
def memoize(fn):
    cache = {}                # closed over
    def wrapper(*args):
        if args not in cache:
            cache[args] = fn(*args)
        return cache[args]
    return wrapper

@memoize
def fib(n):
    if n < 2: return n
    return fib(n-1) + fib(n-2)

print(fib(30))    # 832040  - fast due to caching
```

Q13. What is a partial function and how does it relate to closures?

`functools.partial` creates a new callable with some arguments pre-filled. Internally it stores the function and pre-filled args — similar to a closure but implemented in C. You can replicate it with a closure.

```
from functools import partial

def power(base, exp): return base ** exp

square = partial(power, exp=2)
cube   = partial(power, exp=3)

print(square(5))    # 25
print(cube(3))      # 27

# Closure equivalent
def make_power(exp):
    return lambda base: power(base, exp)

square2 = make_power(2)
print(square2(5))   # 25
```

Q14. How do closures enable data hiding / encapsulation?

Variables in an enclosing scope are not accessible from outside the closure. This provides encapsulation without a class: the data is private, accessible only through the returned functions.

```
def make_account(initial_balance):
    balance = initial_balance    # private
    def deposit(amount):
        nonlocal balance
        balance += amount
```

```

def withdraw(amount):
    nonlocal balance
    if amount > balance: raise ValueError('Insufficient funds')
    balance -= amount
def get_balance(): return balance
return deposit, withdraw, get_balance

dep, wd, bal = make_account(100)
dep(50)
wd(30)
print(bal())    # 120

```

Q15. What is a cell object in Python?

A cell object is a CPython implementation detail that stores a shared reference to a variable used by both an outer and inner function. Both frames hold a pointer to the same cell, so changes made by one are visible to the other.

```

def outer():
    x = 10
    def inner():
        nonlocal x
        x += 1
        return x
    return inner, lambda: x    # both share the same cell for x

inc, get = outer()
print(inc())    # 11
print(get())    # 11 - same cell, sees mutation

```

Q16. How do closures interact with loops and list comprehensions?

List comprehension variables are local to the comprehension (Python 3). A lambda inside a comprehension closes over its own loop variable correctly. Generator expressions have the same scoping.

```

# Python 3: list comp has its own scope
fns = [lambda x=x: x for x in range(5)]
print([f() for f in fns])    # [0, 1, 2, 3, 4]

# Generator expressions - lazy, own scope too
gen_fns = (lambda: i for i in range(5))    # late binding still
# Use default arg trick inside a genexpr:
gen_fns2 = (lambda i=i: i for i in range(5))
print([f() for f in gen_fns2])    # [0, 1, 2, 3, 4]

```

Q17. Can you create closures inside a class?

Yes. Methods that return inner functions form closures over instance or local variables, and those closures persist independently of the instance's lifecycle.

```

class Multiplier:
    def __init__(self, factor):
        self.factor = factor

    def make_op(self, offset):
        factor = self.factor      # capture by value
        def op(x):
            return x * factor + offset
        return op

m = Multiplier(3)
fn = m.make_op(10)
del m                                # instance gone
print(fn(4))    # 22 - closure still holds factor=3

```

Q18. What is the difference between a closure and a lambda?

A lambda is a short, anonymous function expression — it may or may not be a closure. A closure is any function (def or lambda) that captures free variables from an enclosing scope. All lambdas that reference outer variables are closures; not all closures are lambdas.

```

x = 10

# Lambda that IS a closure
fn = lambda y: x + y
print(fn(5))    # 15 - x is free variable

# Lambda that is NOT a closure
pure = lambda a, b: a + b
print(pure.__closure__)    # None

```

Q19. How does garbage collection interact with closures?

The cell objects holding free variables are reference-counted. As long as the closure function object is alive, its cell objects remain alive — preventing the captured objects from being garbage-collected even if the original scope is gone.

```

import gc, weakref

class BigObj: pass

def make_closure():
    obj = BigObj()
    wr = weakref.ref(obj)
    def fn(): return obj    # keeps obj alive
    return fn, wr

fn, wr = make_closure()
gc.collect()
print(wr() is not None)    # True - obj alive via closure
del fn
gc.collect()
print(wr() is None)        # True - obj now freed

```

Q20. What is a closure-based singleton?

A closure over a list or dict can lazily initialise and cache a single instance, providing singleton behaviour without a class.

```
def make_singleton(cls):
    instances = {}
    def get_instance(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]
    return get_instance

class Config:
    def __init__(self, env): self.env = env

get_config = make_singleton(Config)
a = get_config('prod')
b = get_config('dev')    # second call ignored
print(a is b)           # True
print(a.env)            # prod
```

Section 2 — Decorator Fundamentals (Q21–Q50)

Q21. What is a decorator in Python?

A decorator is a callable that takes a function (or class) as an argument, wraps it with additional behaviour, and returns a new callable. The `@` syntax is syntactic sugar for function reassignment.

```
def shout(fn):
    def wrapper(*args, **kwargs):
        result = fn(*args, **kwargs)
        return str(result).upper()
    return wrapper

@shout
def greet(name):
    return f'hello {name}'

print(greet('alice'))    # HELLO ALICE
```

Q22. What does `functools.wraps` do and why is it important?

`functools.wraps` copies `__name__`, `__doc__`, `__module__`, `__qualname__`, `__annotations__`, and `__wrapped__` from the wrapped function to the wrapper. Without it, introspection tools see the wrapper's metadata instead of the original function's.

```

from functools import wraps

def my_decorator(fn):
    @wraps(fn)          # preserve metadata
    def wrapper(*args, **kwargs):
        return fn(*args, **kwargs)
    return wrapper

@my_decorator
def greet(name):
    '''Say hello.'''
    return f'Hello, {name}'

print(greet.__name__)    # 'greet' (not 'wrapper')
print(greet.__doc__)     # 'Say hello.'
print(greet.__wrapped__) # original function

```

Q23. How do you write a decorator that accepts arguments (parametrized decorator)?

Add an extra layer: a function that takes the decorator's arguments and returns the actual decorator. This gives a three-level nesting: argument-taker → decorator → wrapper.

```

from functools import wraps

def repeat(times):
    def decorator(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            for _ in range(times):
                result = fn(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(3)
def say(msg):
    print(msg)

say('hi')    # hi hi hi

```

Q24. How do you stack (chain) multiple decorators?

Python applies stacked decorators bottom-up. @A @B def fn means fn = A(B(fn)). The outermost decorator is applied last but its wrapper runs first when the function is called.

```

def bold(fn):
    def wrapper(*a, **kw): return f'<b>{fn(*a, **kw)}</b>'
    return wrapper

def italic(fn):
    def wrapper(*a, **kw): return f'<i>{fn(*a, **kw)}</i>'
    return wrapper

@bold

```

```
@italic          # applied first: fn = bold(italic(fn))
def text(s): return s

print(text('hello'))  # <b><i>hello</i></b>
```

Q25. What is the difference between a decorator and a higher-order function?

A higher-order function takes or returns functions — it is a general concept. A decorator is a specific Python pattern/syntax that uses higher-order functions to wrap another function. All decorators are higher-order functions; not all higher-order functions are decorators.

```
# Higher-order function (general)
def apply_twice(fn, x): return fn(fn(x))
print(apply_twice(lambda x: x+1, 5))  # 7

# Decorator (specific pattern)
def log(fn):
    def wrapper(*a, **kw):
        print(f'calling {fn.__name__}')
        return fn(*a, **kw)
    return wrapper

@log
def add(x, y): return x + y
add(1, 2)  # calling add
```

Q26. How do you write a decorator that works with or without arguments?

Check whether the first argument is callable (called without args) or not (called with args). Use `functools.wraps` and handle both call signatures.

```
from functools import wraps

def log(fn=None, *, prefix='LOG'):
    def decorator(f):
        @wraps(f)
        def wrapper(*args, **kwargs):
            print(f'{prefix}: {f.__name__}')
            return f(*args, **kwargs)
        return wrapper
    if fn is not None:  # used as @log (no args)
        return decorator(fn)
    return decorator  # used as @log(...)

@log
def foo(): pass

@log(prefix='DEBUG')
def bar(): pass

foo()  # LOG: foo
bar()  # DEBUG: bar
```

Q27. What is a class-based decorator?

A class whose `__init__` receives the decorated function and whose `__call__` implements the wrapper behaviour. Useful when the decorator needs complex state that is awkward to manage in a closure.

```
from functools import wraps
import time

class Retry:
    def __init__(self, fn, attempts=3):
        self.fn = fn
        self.attempts = attempts
        wraps(fn)(self)

    def __call__(self, *args, **kwargs):
        for i in range(1, self.attempts + 1):
            try:
                return self.fn(*args, **kwargs)
            except Exception as e:
                if i == self.attempts: raise
                print(f'Retry {i}: {e}')

@Retry
def unstable():
    raise IOError('fail')

# unstable() # retries 3 times then raises
```

Q28. How do you write a class-based decorator that takes arguments?

The class `__init__` receives the decorator arguments; `__call__` receives the function to decorate and returns the wrapper. This adds one more level compared to argument-free class decorators.

```
from functools import wraps

class Retry:
    def __init__(self, attempts=3):
        self.attempts = attempts

    def __call__(self, fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            for i in range(1, self.attempts + 1):
                try:
                    return fn(*args, **kwargs)
                except Exception as e:
                    if i == self.attempts: raise
                    print(f'Attempt {i} failed: {e}')
            return wrapper

@Retry(attempts=2)
def flaky(): raise RuntimeError('boom')

# flaky() # Attempt 1 failed: boom -> raises
```

Q29. What is the `@staticmethod` decorator and how does it work?

`@staticmethod` wraps a function in a non-data descriptor that, when accessed via the class or instance, returns the function unchanged — without prepending `self` or `cls`. It is an ordinary function grouped inside a class.

```
class MathUtils:
    @staticmethod
    def add(a, b):
        return a + b

    @staticmethod
    def is_even(n):
        return n % 2 == 0

print(MathUtils.add(3, 4))      # 7 — no instance needed
print(MathUtils().is_even(6))  # True
```

Q30. What is the `@classmethod` decorator?

`@classmethod` wraps a function in a descriptor that passes the class (not the instance) as the first argument (`cls`). Used for alternative constructors and factory methods.

```
class Date:
    def __init__(self, year, month, day):
        self.year, self.month, self.day = year, month, day

    @classmethod
    def from_string(cls, s):
        y, m, d = map(int, s.split('-'))
        return cls(y, m, d)  # works for subclasses too

    @classmethod
    def today(cls):
        import datetime
        t = datetime.date.today()
        return cls(t.year, t.month, t.day)

d = Date.from_string('2025-05-01')
print(d.year, d.month, d.day)  # 2025 5 1
```

Q31. What is the `@property` decorator?

`@property` converts a method into a read-only attribute accessor. Adding `@attr.setter` / `@attr.deleter` provides write and delete support. Internally it creates a data descriptor.

```
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def radius(self): return self._radius
```

```

@radius.setter
def radius(self, v):
    if v < 0: raise ValueError('Negative radius')
    self._radius = v

@property
def area(self):
    import math
    return math.pi * self._radius ** 2

c = Circle(5)
c.radius = 10
print(c.area)      # 314.159...

```

Q32. What is `@functools.lru_cache` and how does it work?

`@lru_cache` memoizes a function's return values in a least-recently-used cache of bounded size. Arguments must be hashable. It adds `cache_info()` and `cache_clear()` methods to the wrapped function.

```

from functools import lru_cache

@lru_cache(maxsize=128)
def fib(n):
    if n < 2: return n
    return fib(n-1) + fib(n-2)

print(fib(50))          # 12586269025 - fast
print(fib.cache_info()) # CacheInfo(hits=48, misses=51, ...)
fib.cache_clear()

```

Q33. What is `@functools.cache` (Python 3.9+)?

`@cache` is equivalent to `@lru_cache(maxsize=None)` — an unbounded cache. It is simpler and slightly faster because it uses a plain dict without LRU bookkeeping.

```

from functools import cache

@cache
def factorial(n):
    return 1 if n == 0 else n * factorial(n-1)

print(factorial(10))    # 3628800
print(factorial.cache_info()) # maxsize=None, currsize=11

```

Q34. What is `@functools.singledispatch`?

`@singledispatch` creates a generic function that dispatches to different implementations based on the type of the first argument. Register alternatives with `@fn.register(type)`.

```

from functools import singledispatch

@singledispatch
def process(arg):
    raise NotImplementedError(f'No impl for {type(arg)}')

@process.register(int)
def _(arg): return f'int doubled: {arg * 2}'

@process.register(str)
def _(arg): return f'str upper: {arg.upper()}'

@process.register(list)
def _(arg): return f'list len: {len(arg)}'

print(process(5))           # int doubled: 10
print(process('hello'))    # str upper: HELLO
print(process([1,2,3]))    # list len: 3

```

Q35. How do you write a logging decorator?

A logging decorator wraps the function, logging the call arguments, return value, and execution time. Using `functools.wraps` ensures the original metadata is preserved.

```

from functools import wraps
import logging, time

logging.basicConfig(level=logging.DEBUG)

def logged(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        logging.debug(f'CALL {fn.__name__} args={args} kwargs={kwargs}')
        t0 = time.perf_counter()
        result = fn(*args, **kwargs)
        elapsed = time.perf_counter() - t0
        logging.debug(f'DONE {fn.__name__} -> {result!r} ({elapsed:.4f}s)')
        return result
    return wrapper

@logged
def add(a, b): return a + b

add(3, 4)  # DEBUG: CALL add args=(3,4) ...

```

Q36. How do you write a timing decorator?

Measure elapsed time using `time.perf_counter()` before and after the function call. Print or log the delta.

```

from functools import wraps
import time

def timeit(fn):
    @wraps(fn)

```

```

def wrapper(*args, **kwargs):
    start = time.perf_counter()
    result = fn(*args, **kwargs)
    end = time.perf_counter()
    print(f'{fn.__name__}: {end - start:.6f}s')
    return result
return wrapper

@timeit
def slow_sum(n):
    return sum(range(n))

slow_sum(10_000_000)    # slow_sum: 0.2341s

```

Q37. How do you write a retry decorator with exponential backoff?

Catch specified exceptions, sleep for progressively longer intervals, and re-raise after a maximum number of attempts.

```

from functools import wraps
import time

def retry(exception=(Exception,), attempts=3, backoff=1.0):
    def decorator(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            delay = backoff
            for attempt in range(1, attempts + 1):
                try:
                    return fn(*args, **kwargs)
                except exceptions as e:
                    if attempt == attempts: raise
                    print(f'Retry {attempt} in {delay}s: {e}')
                    time.sleep(delay)
                    delay *= 2
            return wrapper
        return decorator

    @retry(exception=(IOError,), attempts=3, backoff=0.01)
    def fetch(): raise IOError('timeout')

    # fetch() # retries then raises

```

Q38. How do you write a type-checking decorator?

Inspect function annotations and validate argument types at call time using `isinstance()`. Raise `TypeError` with a descriptive message on mismatch.

```

from functools import wraps
import inspect

def typed(fn):
    hints = fn.__annotations__
    @wraps(fn)
    def wrapper(*args, **kwargs):

```

```

    bound = inspect.signature(fn).bind(*args, **kwargs)
    bound.apply_defaults()
    for name, val in bound.arguments.items():
        if name in hints and not isinstance(val, hints[name]):
            raise TypeError(
                f'{name} must be {hints[name].__name__}, '
                f'got {type(val).__name__}')
    return fn(*args, **kwargs)
return wrapper

```

```

@typed
def add(x: int, y: int) -> int: return x + y

print(add(1, 2))      # 3
# add(1, '2')         # TypeError: y must be int, got str

```

Q39. How do you write a rate-limiting decorator?

Track the last call time in a closure. If the interval since the last call is less than the minimum period, sleep for the remainder.

```

from functools import wraps
import time

def rate_limit(calls_per_second):
    min_interval = 1.0 / calls_per_second
    def decorator(fn):
        last_called = [0.0]      # list to allow mutation in closure
        @wraps(fn)
        def wrapper(*args, **kwargs):
            elapsed = time.monotonic() - last_called[0]
            if elapsed < min_interval:
                time.sleep(min_interval - elapsed)
            last_called[0] = time.monotonic()
            return fn(*args, **kwargs)
        return wrapper
    return decorator

@rate_limit(2)    # max 2 calls/second
def ping(): print('ping', time.monotonic())

for _ in range(3): ping() # ~0.5s apart

```

Q40. How do you write a caching decorator with TTL (time-to-live)?

Store cached values alongside their expiry timestamps. On each call, check if the cached value is still valid.

```

from functools import wraps
import time

def ttl_cache(seconds):
    def decorator(fn):
        cache = {}
        @wraps(fn)

```

```

def wrapper(*args):
    now = time.monotonic()
    if args in cache:
        value, expiry = cache[args]
        if now < expiry:
            return value
    result = fn(*args)
    cache[args] = (result, now + seconds)
    return result
return wrapper
return decorator

```

```

@ttl_cache(seconds=2)
def get_data(key):
    print(f'Fetching {key}')
    return f'data_{key}'

```

```

get_data('x')    # Fetching x
get_data('x')    # cached
time.sleep(2.1)
get_data('x')    # Fetching x (expired)

```

Q41. What is functools.wraps doing to __wrapped__?

functools.wraps sets __wrapped__ on the wrapper to point to the original function. This allows unwrap() and introspection tools to follow the chain of decorators back to the original callable.

```

from functools import wraps

def dec(fn):
    @wraps(fn)
    def wrapper(*a, **kw): return fn(*a, **kw)
    return wrapper

@dec
def original(): pass

print(original.__wrapped__ is original.__wrapped__) # True

import inspect
print(inspect.unwrap(original).__name__) # 'original'

```

Q42. How do you write an authentication decorator?

Check credentials before allowing the function to execute. Raise an exception or return an error response if authentication fails.

```

from functools import wraps

def requires_auth(fn):
    @wraps(fn)
    def wrapper(request, *args, **kwargs):
        if not request.get('authenticated'):
            raise PermissionError('Authentication required')
    return wrapper

```

```

        return fn(request, *args, **kwargs)
    return wrapper

@requires_auth
def get_profile(request):
    return f"Profile for {request['user']}"

req = {'authenticated': True, 'user': 'alice'}
print(get_profile(req))  # Profile for alice

# get_profile({'authenticated': False}) # PermissionError

```

Q43. How do you write a deprecated decorator?

Emit a `DeprecationWarning` with `warnings.warn` before calling the function. Optionally accept a message and alternative function name.

```

import warnings
from functools import wraps

def deprecated(reason=''):
    def decorator(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            msg = f'{fn.__name__} is deprecated. {reason}'
            warnings.warn(msg, DeprecationWarning, stacklevel=2)
            return fn(*args, **kwargs)
        return wrapper
    return decorator

@deprecated('Use new_api() instead.')
def old_api():
    return 'old result'

old_api()  # DeprecationWarning: old_api is deprecated ...

```

Q44. How do you create a decorator that adds methods to a class?

A class decorator receives the class object, modifies it (adds attributes, wraps methods), and returns it.

```

def add_repr(cls):
    def __repr__(self):
        attrs = ', '.join(f'{k}={v!r}' for k, v in self.__dict__.items())
        return f'{cls.__name__}({attrs})'
    cls.__repr__ = __repr__
    return cls

@add_repr
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

print(Point(1, 2))  # Point(x=1, y=2)

```

Q45. What is the difference between decorating a function and decorating a class?

Function decorators wrap a single callable. Class decorators receive the entire class object and can modify its methods, add new ones, wrap `__init__`, register it, or replace it entirely. `@dataclass` is a well-known class decorator.

```
import time

def timeit_all_methods(cls):
    from functools import wraps
    for name, method in vars(cls).items():
        if callable(method) and not name.startswith('_'):
            def timed(fn):
                @wraps(fn)
                def wrapper(*a, **kw):
                    t = time.perf_counter()
                    r = fn(*a, **kw)
                    print(f'{fn.__name__}: {time.perf_counter()-t:.4f}s')
                    return r
                return wrapper
            setattr(cls, name, timed(method))
    return cls

@timeit_all_methods
class MyService:
    def run(self): return sum(range(10**6))

MyService().run()    # run: 0.0421s
```

Q46. What is `@dataclass` and how does it work as a decorator?

`@dataclass` inspects class annotations and auto-generates `__init__`, `__repr__`, `__eq__`, and optionally `__hash__`, `__lt__`, `__frozen__` methods. It modifies the class in place and returns it.

```
from dataclasses import dataclass, field

@dataclass(order=True, frozen=True)
class Point:
    x: float
    y: float
    label: str = field(default='', compare=False)

p1 = Point(1.0, 2.0)
p2 = Point(1.0, 2.0, 'origin')
print(p1 == p2)    # True (label excluded)
print(p1 < Point(3.0, 0.0))    # True
print(hash(p1))    # hashable – frozen=True
```

Q47. How does `@abstractmethod` work internally?

@abstractmethod sets `__isabstractmethod__ = True` on the method. ABCMeta's `__new__` collects all methods with this flag into `__abstractmethods__`. If `__abstractmethods__` is non-empty, instantiation raises `TypeError`.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self) -> float: ...

    @abstractmethod
    def perimeter(self) -> float: ...

print(Shape.area.__isabstractmethod__) # True
# Shape() # TypeError: Can't instantiate abstract class

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14159 * self.r ** 2
    def perimeter(self): return 2 * 3.14159 * self.r

c = Circle(5) # OK
```

Q48. How do you decorate an async function?

The wrapper must also be `async` (`async def`) and use `await` when calling the original coroutine. Otherwise the coroutine is never awaited.

```
from functools import wraps
import asyncio, time

def async_timeit(fn):
    @wraps(fn)
    async def wrapper(*args, **kwargs): # must be async
        t0 = time.perf_counter()
        result = await fn(*args, **kwargs) # must await
        print(f'{fn.__name__}: {time.perf_counter()-t0:.4f}s')
        return result
    return wrapper

@async_timeit
async def fetch():
    await asyncio.sleep(0.1)
    return 'data'

asyncio.run(fetch()) # fetch: 0.1002s
```

Q49. What is the `contextlib.contextmanager` decorator?

@contextmanager turns a generator function with one `yield` into a context manager without writing a class. Code before `yield` runs as `__enter__`; code after `yield` runs as `__exit__`.

```
from contextlib import contextmanager
```

```

@contextmanager
def managed_resource(name):
    print(f'Acquiring {name}')
    resource = {'name': name, 'active': True}
    try:
        yield resource
    except Exception as e:
        print(f'Error: {e}')
        raise
    finally:
        resource['active'] = False
        print(f'Releasing {name}')

with managed_resource('DB') as r:
    print(f'Using {r}')
# Acquiring DB -> Using ... -> Releasing DB

```

Q50. How do you preserve the signature of a decorated function?

`functools.wraps` preserves `__wrapped__` and metadata but not the callable signature itself in all tools. For full signature preservation (e.g. for IDEs and Sphinx), use the decorator library or explicitly re-declare with `inspect.Signature`.

```

from functools import wraps
import inspect

def my_dec(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        return fn(*args, **kwargs)
    # Copy signature explicitly
    wrapper.__signature__ = inspect.signature(fn)
    return wrapper

@my_dec
def add(x: int, y: int = 0) -> int: return x + y

print(inspect.signature(add)) # (x: int, y: int = 0) -> int

```

Section 3 — Advanced Decorator Patterns (Q51–Q75)

Q51. What is a descriptor and how does it relate to decorators?

A descriptor is an object implementing `__get__`, `__set__`, or `__delete__`. `staticmethod`, `classmethod`, and `property` are all descriptors. When Python looks up an attribute, descriptors intercept the access — this is the underlying mechanism that makes those decorators work.

```

class Validator:
    def __set_name__(self, owner, name): self.name = name
    def __get__(self, obj, objtype=None):

```

```

        if obj is None: return self
        return obj.__dict__.get(self.name)
    def __set__(self, obj, value):
        if not isinstance(value, (int, float)):
            raise TypeError(f'{self.name} must be numeric')
        obj.__dict__[self.name] = value

class Vector:
    x = Validator()
    y = Validator()
    def __init__(self, x, y): self.x = x; self.y = y

v = Vector(1.0, 2.0)
# v.x = 'bad' # TypeError

```

Q52. How do you implement a decorator registry?

Use a dict in the module or class scope to map names/types to decorated functions. The decorator adds the function to the registry and returns it unchanged (or wrapped).

```

REGISTRY = {}

def register(name):
    def decorator(fn):
        REGISTRY[name] = fn
        return fn # unmodified
    return decorator

@register('add')
def add(x, y): return x + y

@register('mul')
def mul(x, y): return x * y

print(REGISTRY['add'](2, 3)) # 5
print(REGISTRY['mul'](2, 3)) # 6

```

Q53. How do you write a thread-safe caching decorator?

Protect the cache dict with a threading.Lock. Acquire the lock before reading or writing the cache to prevent race conditions.

```

from functools import wraps
import threading

def thread_safe_cache(fn):
    cache = {}
    lock = threading.Lock()
    @wraps(fn)
    def wrapper(*args):
        with lock:
            if args in cache:
                return cache[args]
        result = fn(*args) # compute outside lock
    return wrapper

```

```

        with lock:
            cache[args] = result
        return result
    return wrapper

@thread_safe_cache
def expensive(n): return n ** 2

print(expensive(10))    # 100

```

Q54. How do you implement a once-only decorator (run at most once)?

Store the result after the first call. Return the stored result on subsequent calls without re-executing the function.

```

from functools import wraps

def run_once(fn):
    UNSET = object()
    result = [UNSET]
    @wraps(fn)
    def wrapper(*args, **kwargs):
        if result[0] is UNSET:
            result[0] = fn(*args, **kwargs)
        return result[0]
    return wrapper

@run_once
def load_config():
    print('Loading config...')
    return {'debug': True}

print(load_config())    # Loading config... {'debug': True}
print(load_config())    # {'debug': True} - no reload

```

Q55. How do you chain decorators preserving the order of execution?

Decorator application is bottom-up (inner first), but wrapper execution is top-down (outer first). Understanding this order is essential for decorators that depend on each other (e.g., auth before rate-limit).

```

def first(fn):
    def w(*a, **kw): print('first-before'); r=fn(*a, **kw); print('first-after');
    return r
    return w

def second(fn):
    def w(*a, **kw): print('second-before'); r=fn(*a, **kw); print('second-after');
    return r
    return w

@first
@second
def fn(): print('fn')

```

```
fn()
# first-before second-before fn second-after first-after
```

Q56. What is `__call__` and how does it enable callable objects as decorators?

Any object with `__call__` can be used as a decorator. This allows decorators to maintain rich state (multiple methods, serialisable, testable) while looking like functions at the call site.

```
class CountCalls:
    def __init__(self, fn):
        self.fn = fn
        self.count = 0
        from functools import wraps
        wraps(fn)(self)

    def __call__(self, *args, **kwargs):
        self.count += 1
        return self.fn(*args, **kwargs)

@CountCalls
def add(x, y): return x + y

add(1, 2); add(3, 4)
print(add.count) # 2
```

Q57. How do you write a decorator that works on both functions and methods?

When a plain-function decorator is applied to a method, `self` is just the first positional argument — it works transparently. However, if you need to detect whether the decorated object is a method, use descriptors.

```
from functools import wraps

def logged(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        print(f'Calling {fn.__qualname__}')
        return fn(*args, **kwargs)
    return wrapper

@logged
def standalone(x): return x

class MyClass:
    @logged
    def method(self, x): return x

standalone(1) # Calling standalone
MyClass().method(2) # Calling MyClass.method
```

Q58. How does contextlib.wraps differ from functools.wraps?

There is no contextlib.wraps — functools.wraps is the standard tool. contextlib uses functools.wraps internally inside @contextmanager to preserve the wrapped generator function's metadata on the resulting context manager.

```
from contextlib import contextmanager
from functools import wraps

@contextmanager
def my_ctx():
    '''My context manager.'''
    yield

print(my_ctx.__name__)    # 'my_ctx'
print(my_ctx.__doc__)     # 'My context manager.'
# contextlib uses wraps internally to achieve this
```

Q59. What is the difference between a decorator and monkey-patching?

Both modify behaviour at runtime. A decorator is applied at definition time with @syntax and is explicit, composable, and readable. Monkey-patching replaces an attribute on an existing class/module at runtime, often from outside — it is implicit and harder to track.

```
# Decorator – explicit at definition
def shout(fn):
    def w(*a, **kw): return fn(*a, **kw).upper()
    return w

@shout
def greet(n): return f'hello {n}'

# Monkey-patch – implicit at runtime
class Foo:
    def bar(self): return 'bar'

original = Foo.bar
Foo.bar = lambda self: original(self).upper() # patch
print(Foo().bar())    # BAR
```

Q60. How do you test a decorated function?

Access the original function via __wrapped__ (set by functools.wraps) to test it in isolation. Test the decorated version separately to verify decorator behaviour.

```
from functools import wraps

def validate_positive(fn):
    @wraps(fn)
    def wrapper(x):
        if x < 0: raise ValueError('Must be positive')
        return fn(x)
    return wrapper
```

```

@validate_positive
def sqrt(x): return x ** 0.5

# Test original logic
print(sqrt.__wrapped__(4))    # 2.0 – no validation

# Test decorator
try:
    sqrt(-1)
except ValueError as e:
    print(e)    # Must be positive

```

Q61. What is `inspect.unwrap()` and when do you use it?

`inspect.unwrap(fn)` follows the `__wrapped__` chain set by `functools.wraps` until it finds the original function. Useful for test frameworks, documentation generators, and debuggers that need to inspect the true underlying callable.

```

import inspect
from functools import wraps

def dec_a(fn):
    @wraps(fn)
    def wrapper(*a, **kw): return fn(*a, **kw)
    return wrapper

def dec_b(fn):
    @wraps(fn)
    def wrapper(*a, **kw): return fn(*a, **kw)
    return wrapper

@dec_a
@dec_b
def original(x): return x

print(original.__name__)          # 'original'
print(inspect.unwrap(original) is original.__wrapped__.__wrapped__)
# True – unwrap follows the full chain

```

Q62. How do you implement method decorators inside a class?

Define the decorator as a static method or as a standalone function and apply it with `@`. It receives the method as a plain function (`self` is just the first arg) and works identically.

```

class Service:
    @staticmethod
    def validate(fn):
        from functools import wraps
        @wraps(fn)
        def wrapper(self, x):
            if not isinstance(x, int):
                raise TypeError('Expected int')
            return fn(self, x)
        return wrapper

```

```

    @validate
    def double(self, x): return x * 2

s = Service()
print(s.double(5))    # 10
# s.double('a')      # TypeError

```

Q63. What is `__init_subclass__` and how is it a class decorator alternative?

`__init_subclass__` is called on the parent class whenever it is subclassed, receiving the child class. It can modify the child class in place — effectively acting as an automatic class decorator applied to all subclasses.

```

class Validated:
    def __init_subclass__(cls, **kwargs):
        super().__init_subclass__(**kwargs)
        # Auto-register all subclasses
        if not hasattr(Validated, '_registry'):
            Validated._registry = {}
        Validated._registry[cls.__name__] = cls

class Dog(Validated): pass
class Cat(Validated): pass

print(Validated._registry)
# {'Dog': <class 'Dog'>, 'Cat': <class 'Cat'>}

```

Q64. How does `@functools.total_ordering` work?

`@total_ordering` is a class decorator. Given `__eq__` and one of `__lt__`, `__le__`, `__gt__`, or `__ge__`, it fills in the remaining comparison methods by deriving them logically.

```

from functools import total_ordering

@total_ordering
class Version:
    def __init__(self, major, minor):
        self.major, self.minor = major, minor

    def __eq__(self, other):
        return (self.major, self.minor) == (other.major, other.minor)

    def __lt__(self, other):
        return (self.major, self.minor) < (other.major, other.minor)

v1, v2 = Version(1, 0), Version(2, 0)
print(v1 < v2)      # True
print(v1 >= v2)     # False — derived by total_ordering
print(v1 <= v1)     # True

```

Q65. How do you attach metadata to a decorator for introspection?

Set custom attributes on the wrapper function inside the decorator. Since `functools.wraps` sets `__wrapped__`, you can add your own attributes without conflict.

```
from functools import wraps

def route(path, methods=('GET',)):
    def decorator(fn):
        @wraps(fn)
        def wrapper(*a, **kw): return fn(*a, **kw)
        wrapper.route_path = path
        wrapper.route_methods = methods
        return wrapper
    return decorator

@route('/api/users', methods=('GET', 'POST'))
def users_view(request): pass

print(users_view.route_path)      # /api/users
print(users_view.route_methods)   # ('GET', 'POST')
```

Q66. What is the difference between `@decorator` and `decorator(fn)` manually?

They are identical. `@decorator` above a `def` is purely syntactic sugar for `fn = decorator(fn)` executed immediately after the `def` statement. There is no runtime difference.

```
def shout(fn):
    def w(*a, **kw): return fn(*a, **kw).upper()
    return w

# Sugar
@shout
def greet1(n): return f'hello {n}'

# Manual equivalent
def greet2(n): return f'hello {n}'
greet2 = shout(greet2)

print(greet1('alice'))    # HELLO ALICE
print(greet2('alice'))    # HELLO ALICE
```

Q67. How do you write a decorator that modifies the return value?

Call the original function, transform its result, and return the transformed value from the wrapper.

```
from functools import wraps

def as_json(fn):
    import json
    @wraps(fn)
    def wrapper(*args, **kwargs):
        result = fn(*args, **kwargs)
        return json.dumps(result, indent=2)
    return wrapper
```

```
@as_json
def get_user(uid):
    return {'id': uid, 'name': 'Alice', 'active': True}

print(get_user(42))
# {
#   "id": 42,
#   "name": "Alice",
#   "active": true
# }
```

Q68. How do you write a decorator that modifies the input arguments?

Inspect and transform args/kwargs in the wrapper before passing them to the original function.

```
from functools import wraps

def lowercase_args(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        new_args = tuple(a.lower() if isinstance(a, str) else a for a in args)
        new_kwargs = {k: v.lower() if isinstance(v, str) else v
                      for k, v in kwargs.items()}
        return fn(*new_args, **new_kwargs)
    return wrapper

@lowercase_args
def greet(first, last): return f'Hello, {first} {last}'

print(greet('ALICE', 'SMITH'))    # Hello, alice smith
```

Q69. What is a sentinel decorator and what is it used for?

A sentinel decorator marks a function with a special attribute so that other code can identify it. It does not change the function's behaviour, only its metadata.

```
def requires_permission(permission):
    def decorator(fn):
        fn._required_permission = permission
        return fn          # unchanged!
    return decorator

@requires_permission('admin')
def delete_user(uid): pass

# Framework code inspects the attribute:
def check_and_call(fn, user_permissions, *args):
    perm = getattr(fn, '_required_permission', None)
    if perm and perm not in user_permissions:
        raise PermissionError(f'Need {perm}')
    return fn(*args)

check_and_call(delete_user, {'admin'}, 1)    # OK
```

Q70. How do you implement a decorator that profiles memory usage?

Use tracemalloc to snapshot memory before and after the function call and report the peak allocation.

```
from functools import wraps
import tracemalloc

def profile_memory(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        tracemalloc.start()
        result = fn(*args, **kwargs)
        current, peak = tracemalloc.get_traced_memory()
        tracemalloc.stop()
        print(f'{fn.__name__} peak memory: {peak/1024:.2f} KB')
        return result
    return wrapper

@profile_memory
def build_list(n): return list(range(n))

build_list(100_000)    # build_list peak memory: ~800 KB
```

Q71. How do you apply a decorator to every method in a class automatically?

Write a class decorator that iterates over the class dict, wraps every callable that isn't dunder, and sets the wrapped version back on the class.

```
from functools import wraps

def log_all_methods(cls):
    for name in list(vars(cls)):
        attr = getattr(cls, name)
        if callable(attr) and not name.startswith('__'):
            def make_logged(fn):
                @wraps(fn)
                def logged(*a, **kw):
                    print(f'>> {fn.__qualname__}')
                    return fn(*a, **kw)
                return logged
            setattr(cls, name, make_logged(attr))
    return cls

@log_all_methods
class Calc:
    def add(self, x, y): return x + y
    def mul(self, x, y): return x * y

c = Calc()
c.add(1, 2)    # >> Calc.add
```

Q72. How do decorators interact with class inheritance?

Decorators applied to methods in a parent class wrap the method object, not the class slot. A subclass that overrides the method gets an undecorated version. Re-apply the decorator in the subclass if needed, or use `__init_subclass__`.

```
from functools import wraps

def verbose(fn):
    @wraps(fn)
    def w(self, *a, **kw):
        print(f'{type(self).__name__}.{fn.__name__} called')
        return fn(self, *a, **kw)
    return w

class Parent:
    @verbose
    def greet(self): return 'hi'

class Child(Parent):
    def greet(self): return 'hey'    # not verbose

Parent().greet()    # Parent.greet called
Child().greet()    # no output – override bypasses decorator
```

Q73. What is a parametrized class decorator?

A function that takes parameters and returns a class decorator. The extra function call creates the class decorator with the parameters captured in its closure.

```
def singleton(strict=True):
    def decorator(cls):
        instances = {}
        original_new = cls.__new__
        def __new__(klass, *a, **kw):
            if klass not in instances:
                instances[klass] = original_new(klass)
            elif strict:
                raise RuntimeError('Singleton already created')
            return instances[klass]
        cls.__new__ = __new__
        return cls
    return decorator

@singleton(strict=False)
class Config:
    pass

a = Config(); b = Config()
print(a is b)    # True
```

Q74. How do you create a decorator that enforces function contracts (pre/post conditions)?

Check preconditions before calling the function and postconditions on the result. Raise `AssertionError` or a custom exception on violation.

```
from functools import wraps

def contract(pre=None, post=None):
    def decorator(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            if pre and not pre(*args, **kwargs):
                raise ValueError(f'Precondition failed for {fn.__name__}')
            result = fn(*args, **kwargs)
            if post and not post(result):
                raise ValueError(f'Postcondition failed for {fn.__name__}')
            return result
        return wrapper
    return decorator

@contract(
    pre = lambda x, y: x >= 0 and y >= 0,
    post = lambda r: r >= 0
)
def safe_divide(x, y):
    return x / y if y else 0

print(safe_divide(10, 2))    # 5.0
# safe_divide(-1, 2)        # ValueError: Precondition failed
```

Q75. How do you write a decorator that converts exceptions?

Catch specific exception types in the wrapper and re-raise them as different exception types, preserving the original traceback with `raise ... from`.

```
from functools import wraps

def convert_exceptions(source, target):
    def decorator(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            try:
                return fn(*args, **kwargs)
            except source as e:
                raise target(str(e)) from e
        return wrapper
    return decorator

@convert_exceptions(KeyError, ValueError)
def get_value(d, key):
    return d[key]

try:
    get_value({'a': 1}, 'b')
except ValueError as e:
    print(f'ValueError: {e}')    # ValueError: 'b'
```

Section 4 — Real-World Patterns & Tricky Questions (Q76–Q100)

Q76. How do Flask routes work as decorators?

Flask's `@app.route` is a parametrized decorator. It registers the view function in the app's URL map and returns the function unchanged. The decorator is a sentinel that side-effects the registry.

```
# Flask-style route decorator (simplified)
class App:
    def __init__(self): self._routes = {}

    def route(self, path, methods=('GET',)):
        def decorator(fn):
            self._routes[(path, tuple(methods))] = fn
            return fn # unchanged – just registers
        return decorator

    def dispatch(self, path, method='GET'):
        fn = self._routes.get((path, (method,)))
        return fn() if fn else '404'

app = App()

@app.route('/hello')
def hello(): return 'Hello World'

print(app.dispatch('/hello')) # Hello World
```

Q77. How do pytest fixtures work as decorators?

`@pytest.fixture` marks a function as a fixture. pytest's collection mechanism inspects test function signatures, matches parameter names to fixture names, and injects them — the decorator just registers the function in pytest's registry.

```
# Simplified fixture decorator
FIXTURES = {}

def fixture(fn):
    FIXTURES[fn.__name__] = fn
    return fn

def inject_fixtures(test_fn):
    import inspect
    params = inspect.signature(test_fn).parameters
    kwargs = {name: FIXTURES[name]() for name in params if name in FIXTURES}
    return lambda: test_fn(**kwargs)

@fixture
def db(): return {'host': 'localhost'}

def test_connect(db):
    assert db['host'] == 'localhost'

inject_fixtures(test_connect)() # runs test with db injected
```

Q78. What is the decorator pattern in the Gang of Four (GoF) sense vs Python decorators?

GoF Decorator is an OOP structural pattern that wraps an object in another object implementing the same interface to add behaviour. Python @decorators are a syntactic feature that wraps any callable. Python decorators are often inspired by GoF but are more general and syntactically lighter.

```
# GoF Decorator in Python (class-based wrapping same interface)
class Logger:
    def __init__(self, component): self._c = component
    def operation(self):
        print('LOG:', self._c.__class__.__name__)
        return self._c.operation()

class Core:
    def operation(self): return 'core result'

result = Logger(Core()).operation()
print(result)  # LOG: Core  core result
```

Q79. How do you use decorators to implement aspect-oriented programming (AOP)?

Decorators encapsulate cross-cutting concerns (logging, security, transactions) in one place and apply them declaratively — the core idea of AOP.

```
from functools import wraps

# Cross-cutting: transaction management
def transactional(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        print('[TX] BEGIN')
        try:
            result = fn(*args, **kwargs)
            print('[TX] COMMIT')
            return result
        except Exception as e:
            print(f'[TX] ROLLBACK: {e}')
            raise
    return wrapper

@transactional
def transfer(from_acc, to_acc, amount):
    print(f'Transferring {amount}')

transfer('A', 'B', 100)
# [TX] BEGIN  Transferring 100  [TX] COMMIT
```

Q80. How do you implement a decorator that supports both sync and async functions?

Check with `inspect.iscoroutinefunction` whether the decorated function is async. Return an async wrapper for coroutines and a sync wrapper for regular functions.

```
from functools import wraps
import inspect, asyncio, time

def timeit(fn):
    if inspect.iscoroutinefunction(fn):
        @wraps(fn)
        async def async_wrapper(*a, **kw):
            t = time.perf_counter()
            r = await fn(*a, **kw)
            print(f'{fn.__name__}: {time.perf_counter()-t:.4f}s')
            return r
        return async_wrapper
    @wraps(fn)
    def sync_wrapper(*a, **kw):
        t = time.perf_counter()
        r = fn(*a, **kw)
        print(f'{fn.__name__}: {time.perf_counter()-t:.4f}s')
        return r
    return sync_wrapper

@timeit
def sync_work(): return sum(range(10**6))

@timeit
async def async_work(): await asyncio.sleep(0.05)

sync_work()
asyncio.run(async_work())
```

Q81. What happens to a closure's free variables when the outer function is called multiple times?

Each call to the outer function creates a new closure with a fresh binding of the free variables — independent from all previous calls. The cell objects are newly allocated per call.

```
def make_counter(start=0):
    count = start
    def inc():
        nonlocal count
        count += 1
        return count
    return inc

c1 = make_counter(0)
c2 = make_counter(100)

print(c1())    # 1
print(c1())    # 2
print(c2())    # 101 — independent
print(c1())    # 3 — unaffected by c2
```

Q82. How do you write a decorator that limits the number of times a function can be called?

Track call count in the closure. Raise an error (or return a default) when the limit is exceeded.

```
from functools import wraps

def max_calls(n):
    def decorator(fn):
        calls = [0]
        @wraps(fn)
        def wrapper(*args, **kwargs):
            calls[0] += 1
            if calls[0] > n:
                raise RuntimeError(f'{fn.__name__} exceeded {n} calls')
            return fn(*args, **kwargs)
        return wrapper
    return decorator

@max_calls(3)
def ping(): return 'pong'

print(ping()) # pong
print(ping()) # pong
print(ping()) # pong
# ping()      # RuntimeError: ping exceeded 3 calls
```

Q83. How do you implement a plugin system using decorators?

A class-level or module-level registry dict holds plugins. The decorator registers the decorated function/class and returns it unchanged.

```
class PluginRegistry:
    _plugins = {}

    @classmethod
    def register(cls, name):
        def decorator(fn):
            cls._plugins[name] = fn
            return fn
        return decorator

    @classmethod
    def get(cls, name): return cls._plugins[name]

@PluginRegistry.register('csv')
def parse_csv(data): return data.split(',')

@PluginRegistry.register('json')
def parse_json(data):
    import json; return json.loads(data)

parser = PluginRegistry.get('csv')
print(parser('a,b,c')) # ['a', 'b', 'c']
```

Q84. What is the difference between `__get__` on a function and on a descriptor?

All Python functions are non-data descriptors. When accessed via a class instance, function.`__get__`(instance, owner) returns a bound method (with self pre-filled). This is how instance methods work — decorators that return plain functions preserve this behaviour; decorators that return non-descriptor objects (like class instances without `__get__`) break method binding.

```
class MethodDecorator:
    def __init__(self, fn): self.fn = fn

    def __get__(self, obj, objtype=None):
        if obj is None: return self
        from functools import partial
        return partial(self, obj) # bind obj

    def __call__(self, obj, *args, **kwargs):
        print(f'Called on {obj}')
        return self.fn(obj, *args, **kwargs)

class MyClass:
    @MethodDecorator
    def greet(self): return 'hi'

MyClass().greet() # Called on <MyClass ...> -> hi
```

Q85. How does `@functools singledispatchmethod` work for class methods?

`@singledispatchmethod` (Python 3.8+) applies single dispatch to instance methods, using the type of the first non-self argument for dispatch.

```
from functools import singledispatchmethod

class Processor:
    @singledispatchmethod
    def process(self, arg):
        raise NotImplementedError(f'No handler for {type(arg)}')

    @process.register
    def _(self, arg: int): return f'int: {arg * 2}'

    @process.register
    def _(self, arg: str): return f'str: {arg.upper()}'

p = Processor()
print(p.process(5)) # int: 10
print(p.process('hi')) # str: HI
```

Q86. How do you implement a decorator that freezes function arguments?

Use `functools.partial` internally: the decorator captures positional and keyword defaults and merges them with call-time arguments.

```
from functools import wraps
```

```
def with_defaults(**defaults):
    def decorator(fn):
        @wraps(fn)
        def wrapper(**kwargs):
            merged = {**defaults, **kwargs}
            return fn(**merged)
        return wrapper
    return decorator

@with_defaults(host='localhost', port=5432)
def connect(host, port, dbname):
    return f'postgresql://{host}:{port}/{dbname}'

print(connect(dbname='mydb'))          # localhost:5432/mydb
print(connect(host='db.prod', dbname='x')) # db.prod:5432/x
```

Q87. How do you write a decorator that adds CLI argument parsing to a function?

In a wrapper, parse `sys.argv` using `argparse`, convert the parsed namespace into `kwargs`, and call the original function.

```
import argparse, sys
from functools import wraps

def cli(fn):
    @wraps(fn)
    def wrapper():
        import inspect
        sig = inspect.signature(fn)
        parser = argparse.ArgumentParser(description=fn.__doc__)
        for name, param in sig.parameters.items():
            default = param.default if param.default is not
inspect.Parameter.empty else None
            parser.add_argument(f'--{name}', default=default)
        args = parser.parse_args()
        return fn(**vars(args))
    return wrapper

@cli
def greet(name='world'):
    '''Greet someone.'''
    print(f'Hello, {name}!')

# python script.py --name Alice -> Hello, Alice!
```

Q88. How do you write a decorator that validates JSON input?

Attempt to JSON-parse a string argument; raise `ValueError` with a clear message if it fails.

```
from functools import wraps
import json

def validate_json(arg_index=0):
    def decorator(fn):
        @wraps(fn)
```

```

def wrapper(*args, **kwargs):
    raw = args[arg_index]
    try:
        parsed = json.loads(raw)
    except json.JSONDecodeError as e:
        raise ValueError(f'Invalid JSON at arg {arg_index}: {e}')
    new_args = list(args)
    new_args[arg_index] = parsed
    return fn(*new_args, **kwargs)
return wrapper
return decorator

@validate_json(arg_index=0)
def process(data):
    return data['key']

print(process({'key': 'value'}))    # value

```

Q89. What is the `__set_name__` hook and how does it help descriptor-based decorators?

`__set_name__`(owner, name) is called when a descriptor is assigned inside a class body. It tells the descriptor the attribute name it was assigned to — eliminating the need to pass the name explicitly.

```

class Bounded:
    def __init__(self, lo, hi):
        self.lo, self.hi = lo, hi
        self.name = None

    def __set_name__(self, owner, name):
        self.name = name

    def __get__(self, obj, objtype=None):
        if obj is None: return self
        return obj.__dict__.get(self.name)

    def __set__(self, obj, value):
        if not (self.lo <= value <= self.hi):
            raise ValueError(f'{self.name} must be in [{self.lo}, {self.hi}]')
        obj.__dict__[self.name] = value

class Temperature:
    celsius = Bounded(-273.15, 1e6)

t = Temperature()
t.celsius = 25
print(t.celsius)    # 25
# t.celsius = -300  # ValueError

```

Q90. How do you implement a read-only attribute using a decorator?

Use `@property` without a setter. Accessing the attribute calls the getter; any attempt to assign raises `AttributeError`.

```

class Circle:
    def __init__(self, r):
        self._r = r

    @property
    def radius(self): return self._r

    @property
    def area(self):
        import math
        return math.pi * self._r ** 2

c = Circle(5)
print(c.radius)    # 5
print(c.area)      # 78.539...
# c.radius = 10    # AttributeError: can't set attribute
# c.area = 10      # AttributeError: can't set attribute

```

Q91. How do you create a decorator that enforces a calling convention?

Use `inspect.signature` and `bind` to enforce keyword-only or positional calling conventions before delegating to the original function.

```

from functools import wraps
import inspect

def keyword_only(fn):
    @wraps(fn)
    def wrapper(**kwargs):
        # force keyword-only at call site
        sig = inspect.signature(fn)
        bound = sig.bind(**kwargs)
        return fn(*bound.args, **bound.kwargs)
    return wrapper

@keyword_only
def create_user(name, role, active=True):
    return {'name': name, 'role': role, 'active': active}

# create_user('alice', 'admin')          # TypeError - no positional
print(create_user(name='alice', role='admin'))
# {'name': 'alice', 'role': 'admin', 'active': True}

```

Q92. What is the overhead of a decorator and how can you minimise it?

Each decorated function call goes through one extra function call (the wrapper) plus closure variable lookups. For hot paths, minimise wrapper logic, use local variable caching for globals, or use C-extension decorators (like `lru_cache`).

```

from functools import wraps
import time

# Slow: repeated global lookups
def slow_log(fn):
    @wraps(fn)

```

```

def wrapper(*a, **kw):
    print(f'Calling {fn.__name__}') # print is a global lookup
    return fn(*a, **kw)
return wrapper

# Faster: cache globals as local defaults
_print = print
def fast_log(fn):
    name = fn.__name__
    @wraps(fn)
    def wrapper(*a, **kw, _print=_print, _name=name):
        _print(f'Calling {_name}')
        return fn(*a, **kw)
    return wrapper

```

Q93. How do you compose multiple decorators into one?

Apply them in sequence using `functools.reduce`, or write a `compose` utility that chains decorators right-to-left.

```

from functools import reduce

def compose(*decorators):
    '''Apply decorators right-to-left (innermost first).'''
    def decorator(fn):
        return reduce(lambda f, d: d(f), reversed(decorators), fn)
    return decorator

def bold(fn): return lambda *a, **kw: f'<b>{fn(*a, **kw)}</b>'
def italic(fn): return lambda *a, **kw: f'<i>{fn(*a, **kw)}</i>'
def upper(fn): return lambda *a, **kw: fn(*a, **kw).upper()

@compose(bold, italic, upper)
def text(s): return s

print(text('hello')) # <b><i>HELLO</i></b>

```

Q94. How do you write a decorator that caches per instance (not globally)?

Use a descriptor to store the cache on the instance's `__dict__`, so each instance has its own cache independent of others.

```

class instance_lru_cache:
    def __init__(self, fn): self.fn = fn
    def __set_name__(self, owner, name): self.name = name

    def __get__(self, obj, objtype=None):
        if obj is None: return self
        cache = obj.__dict__.setdefault(f'_cache_{self.name}', {})
        def wrapper(*args):
            if args not in cache:
                cache[args] = self.fn(obj, *args)
            return cache[args]
        return wrapper

```

```

class Calc:
    @instance_lru_cache
    def fib(self, n):
        if n < 2: return n
        return self.fib(n-1) + self.fib(n-2)

c1, c2 = Calc(), Calc()
print(c1.fib(30))    # 832040  - own cache
print(c2.fib(30))    # 832040  - independent cache

```

Q95. What is the difference between `functools.lru_cache` and manual memoization?

`lru_cache` is implemented in C, thread-safe, supports maxsize eviction, and adds `cache_info/cache_clear`. Manual memoization via a closure is pure Python, slower, not thread-safe, and unbounded by default, but more customisable.

```

# Manual memoize
def memoize(fn):
    cache = {}
    def wrapper(*args):
        if args not in cache:
            cache[args] = fn(*args)
        return cache[args]
    return wrapper

# lru_cache - faster, bounded, thread-safe
from functools import lru_cache

@lru_cache(maxsize=256)
def fib(n):
    if n < 2: return n
    return fib(n-1) + fib(n-2)

print(fib.cache_info()) # hits, misses, maxsize, currsz

```

Q96. How do you implement a decorator that provides a default return on exception?

Catch specified exceptions in the wrapper and return a configured default value instead of propagating.

```

from functools import wraps

def default_on_error(default, exceptions=(Exception,)):
    def decorator(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            try:
                return fn(*args, **kwargs)
            except exceptions:
                return default
        return wrapper
    return decorator

@default_on_error(default=0, exceptions=(ZeroDivisionError, ValueError))

```

```
def safe_div(a, b):
    return a / b

print(safe_div(10, 2))    # 5.0
print(safe_div(10, 0))    # 0 - no crash
```

Q97. How do you make a decorator that is both a function decorator and a class decorator?

Detect the type of the decorated object inside the decorator and handle functions and classes differently.

```
from functools import wraps
import inspect

def trace(target):
    if inspect.isclass(target):
        for name, method in vars(target).items():
            if callable(method) and not name.startswith('__'):
                setattr(target, name, trace(method))
        return target
    @wraps(target)
    def wrapper(*args, **kwargs):
        print(f'>> {target.__qualname__}')
        return target(*args, **kwargs)
    return wrapper

@trace
def standalone(x): return x * 2

@trace
class MyClass:
    def compute(self, x): return x ** 2

standalone(3)          # >> standalone
MyClass().compute(4)    # >> MyClass.compute
```

Q98. How do you implement a decorator using `__init_subclass__` for automatic registration?

Define `__init_subclass__` on a base class to automatically register every subclass into a dict — a decorator-free alternative to `@register`.

```
class Command:
    _commands = {}

    def __init_subclass__(cls, name=None, **kwargs):
        super().__init_subclass__(**kwargs)
        key = name or cls.__name__.lower()
        Command._commands[key] = cls

    @classmethod
    def run(cls, name, *args):
        return cls._commands[name](*args)
```

```
class Greet(Command, name='greet'):
    def __init__(self, who): self.who = who
    def execute(self): return f'Hello, {self.who}'

cmd = Command.run('greet', 'Alice')
print(cmd.execute())    # Hello, Alice
```

Q99. How do you write a decorator that adds context manager support to a function?

Return a context manager object from the wrapper. `contextlib.contextmanager` is the standard way; you can also use a class implementing `__enter__`/`__exit__`.

```
from contextlib import contextmanager
from functools import wraps

def as_context(fn):
    @wraps(fn)
    @contextmanager
    def wrapper(*args, **kwargs):
        print(f'Entering {fn.__name__}')
        result = fn(*args, **kwargs)
        yield result
        print(f'Exiting {fn.__name__}')
    return wrapper

@as_context
def setup_env(name):
    return {'env': name}

with setup_env('prod') as ctx:
    print('Inside with:', ctx)
# Entering setup_env  Inside with: {'env': 'prod'}  Exiting setup_env
```

Q100. Putting it all together — production-grade decorator stack.

Combine logging, timing, retry, type-checking, and metadata preservation in a single composable stack — demonstrating all key decorator concepts.

```
from functools import wraps, reduce
import time, logging, inspect

logging.basicConfig(level=logging.INFO)
log = logging.getLogger(__name__)

def logged(fn):
    @wraps(fn)
    def w(*a, **kw):
        log.info(f'CALL {fn.__qualname__} args={a} kw={kw}')
        r = fn(*a, **kw)
        log.info(f'DONE {fn.__qualname__} -> {r!r}')
        return r
    return w
```

```

def timed(fn):
    @wraps(fn)
    def w(*a, **kw):
        t = time.perf_counter()
        r = fn(*a, **kw)
        log.info(f'TIME {fn.__qualname__}: {time.perf_counter()-t:.6f}s')
        return r
    return w

def retry(attempts=3):
    def dec(fn):
        @wraps(fn)
        def w(*a, **kw):
            for i in range(1, attempts+1):
                try: return fn(*a, **kw)
                except Exception as e:
                    if i == attempts: raise
                    log.warning(f'Retry {i}: {e}')
            return w
        return dec
    return dec

def compose(*decs):
    def dec(fn): return reduce(lambda f,d: d(f), reversed(decs), fn)
    return dec

@compose(logged, timed, retry(3))
def fetch_data(url: str) -> dict:
    '''Fetch data from a URL.'''
    if 'bad' in url: raise IOError('timeout')
    return {'url': url, 'data': '...'}

print(fetch_data('https://api.example.com/data'))
print(fetch_data.__name__)      # 'fetch_data' - metadata preserved
print(fetch_data.__doc__)      # 'Fetch data from a URL.'
print(inspect.unwrap(fetch_data).__name__) # 'fetch_data'

```